

Assignment: CNN Image Classification with PyTorch

Dataset Options (Choose One)

You must train on a standard dataset and then test with your own camera photos.

1. Fashion Items (FashionMNIST): Train on the standard FashionMNIST dataset.
 - Your Custom Photos: Take photos of a T-shirt, a Sneaker, a Bag, or trousers.
2. Hand Gestures (Rock-Paper-Scissors): Train on a standard RPS dataset (widely available).
 - Custom Photos: Take photos of your own hand making Rock, Paper, and Scissors signs.
3. Vehicles & Animals (CIFAR-10 Subset): Train on CIFAR-10.
 - Your Custom Photos: Take a photo of a Car, a Dog, a Cat, or a Bird.
4. Fruits (Fresh Fruit Dataset): Train on a dataset containing Apples, Bananas, and Oranges.
 - Your Custom Photos: Buy the actual fruit and photograph it on a table.
5. Geometric Shapes: Train on a dataset of Triangles, Squares, and Circles (Drawn or digital).
 - Your Custom Photos: Draw a large shape on paper and photograph it.

1. Build a Full CNN Classification Workflow

You must construct a comprehensive Deep Learning pipeline using PyTorch. Your pipeline must bridge standard dataset training with real-world testing.

- **Project on GitHub:** Storing code, custom data, and the trained model state dictionary (.pth).
- **Automatic data retrieval:** The notebook must automatically pull the standard dataset (via torchvision) and your custom images from your GitHub repository.
- **Data Preprocessing:** Use torchvision.transforms for resizing, normalization, and tensor conversion.
- **Model Architecture:** Build a Convolutional Neural Network (CNN) class inheriting from nn.Module.
- **Real-World Testing:** You must find objects matching your classes, take photos with your phone, and use your trained model to classify them.

2. Perform Essential Data Preprocessing & Custom Data Creation

Image data requires specific tensor formatting. Your pipeline must include:

- **Standard Dataset:** Download the dataset (e.g., FashionMNIST) directly via torchvision.datasets.
 1. Transforms: You must define a transform pipeline to Resize (if needed), Convert to Tensor, and Normalize (mean/std).
- **Custom Data Creation (The "Phone" Task):**
 1. Select 10 distinct items matching your dataset classes (e.g., if using FashionMNIST, find 10 items like a shoe, a shirt, a bag, etc.).
 2. Place them on a plain background (clean floor or table) to minimize noise.
 3. Take a photo of each using your smartphone.
 4. **Important:** Process these images to match the training data format (e.g., if the dataset is Grayscale, convert to Grayscale; If RGB, keep RGB. Resize to the exact pixel dimensions of the training set).

- **DataLoader**: Create DataLoader objects for Training and Validation (e.g., batch_size=64).

3. GitHub Repository Setup (Crucial Step)

You must host your project on GitHub to ensure reproducibility.

- Folder Structure:
 - **dataset/**: Upload your 10 custom smartphone images here (e.g., shirt.jpg, sneaker.jpg).
 - **model/**: After training, save your model's state dict (e.g., 190110.pth) and upload it here.
 - **ID.ipynb**: Your Google Colab notebook. (e.g., **190110.ipynb**)
 - **README.md**: Display your results, plots, and a description of your architecture.
- **Requirement**: I must be able to view the repo and see your custom images and the saved model file.

4. Model Implementation in Google Colab

Notebook must demonstrate the following steps without requiring manual file uploads during runtime:

A. Data Loading (Automation)

- Use `!git clone [your-repo-link]` inside the notebook to download your custom phone images.
- Load the standard training data using `torchvision.datasets`.

B. CNN Model Building (PyTorch)

- Define a class `class CNN(nn.Module)`: that includes:
 - `nn.Conv2d` layers (Convolution).
 - `nn.ReLU()` activations.
 - `nn.MaxPool2d` layers.
 - `nn.Linear` layers (Fully connected).
- Forward Pass: Define the `forward(self, x)` method to outline the data flow.

C. Training Loop

- Loss Function: Use `nn.CrossEntropyLoss`.
- Optimizer: Use `torch.optim.Adam` or `SGD`.
- Loop: Write a loop that iterates through epochs and batches:
 - `optimizer.zero_grad()`
 - `loss.backward()`
 - `optimizer.step()`
- Saving: Save the model using `torch.save(model.state_dict(), 'ID.pth')`.

D. Real-World Prediction

- Load your custom images using `PIL` or `OpenCV`.
- Apply the same transforms used on the training set (Resize -> ToTensor -> Normalize).
- Apply the exact same transforms pipeline used on the training set (including the specific Normalize values).
- Pass the image through the model (ensure model is in `.eval()` mode).
- Apply `torch.softmax` to get probabilities.

5. Include the Following Visuals

Generate and display the following within the notebook:

- **Training History Plots**: Two line graphs showing Loss vs. Epochs and Accuracy vs. Epochs for

both training and validation sets.

- **Confusion Matrix:** A heatmap showing how the model performed on the standard test set.
- **Custom Prediction Gallery:** A grid display of your 10 phone images. Above/Below each image, print the Predicted Class and the Confidence %.
 - Example: "Pred: Sneaker (98.5%)"
- **Visual Error Analysis:** Display 3 random images from the standard test set that the model classified incorrectly, showing the true label versus the predicted label.

6. Submission

- **GitHub Link:** The URL to your public repository containing the custom images, the saved model (.pth), and the .ipynb file.
- **Colab Link:** The direct link to your notebook.
- **Constraint:** I must be able to click the Colab link, select "Run All", and the notebook should:
 1. Clone your repo.
 2. Train the model (or load your saved weights).
 3. Process your custom images.
 4. Output the predictions and confidence scores automatically.

Penalty: Failing to maintain any of these automation instructions will result in a penalty. If I have to manually upload files or debug file paths to make your code run, points will be deducted